

Harrow: Synthesis of Optical Logic Circuits via Harmonic Mean and Integer Partition

Jun-Wei Liang, Iris Hui-Ru Jiang, and Kai-Hsiang Chiu

Graduate Institute of Electronics Engineering

National Taiwan University

Taipei 106319, Taiwan

email: {asfhiol, huiru.jiang, khchiu168}@gmail.com

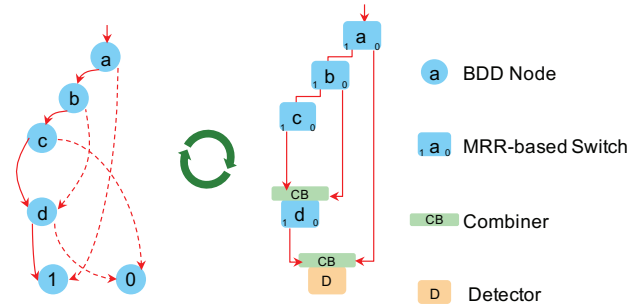
Abstract—With the advancement of high-speed and energy-efficient optical interconnect and computation, photonic integrated circuits (PICs) have become a promising alternative to traditional CMOS circuits. A PIC can be synthesized by mapping the binary decision diagram (BDD) of target functions to optical switches and combiners. However, excessive signal attenuation along the light propagation may require extra optical-electrical signal conversion, thus introducing unwanted delays. In this paper, we aim to overcome this deficiency during logic synthesis: First, we optimize the signal efficiency factor by applying the concept of harmonic means to optimize DC combiners. Second, we eliminate redundant combiners by integer partition. Moreover, we properly arrange these proposed techniques in an optimal sequence of operations to form our main framework. Experimental results show that our framework outperforms the state of the art in terms of efficiency factor.

Index Terms—Photonic integrated circuit, logic synthesis, efficiency factor, harmonic mean, integer partition

I. INTRODUCTION

Signal delay and energy consumption in integrated circuits have become the bottleneck of semiconductor technology advancement. Optical interconnect and computation, leveraging light-based transmission, is viewed as a promising alternative to conventional CMOS circuits, offering benefits in speed and energy efficiency. The automation of logic circuit design for photonic integrated circuits (PICs) has recently garnered considerable research attention [1]–[3]. The design flow of PICs can be divided into logic synthesis, optical component placement, and optical routing considering Wavelength Division Multiplexing (WDM) sharing. As the initial and foundational stage, logic synthesis critically impacts both placement and routing quality in the PIC design process. For instance, the power loss, timing delay, and hardware cost could be primarily determined in the logic synthesis stage [4]–[6].

Condrat *et al.* [4] pioneered the virtual gate concept in optical circuit synthesis, implementing Boolean functions through fundamental logical operations and Binary Decision Diagram (BDD) structures. Furthermore, to prevent significant light splitting loss incurred by [4], Wille *et al.* proposed a reverse BDD-based synthesis approach without using any splitters, which has become the backbone of optical circuit synthesis [5]. As depicted in Fig. 1, a target function is realized by implementing all of the paths from the root node of its BDD to the 1-terminal. Each BDD decision node is mapped to an optical switch controlled by its corresponding variable value, a node with multiple incoming signal paths requires a combiner, and the 1-terminal is replaced by a photo detector. The light source feeds to the top switch (the root node), traverses each switch (BDD node) and edge on the 1-paths, and finally arrives at the photo detector as the function output. Excessive signal attenuation during light propagation, however, can render the



BDD's Function: $a' + ab'd + abcd$

Fig. 1: Conversion between a binary decision diagram (BDD) and a synthesized photonic integrated circuit (PIC).

final output undetectable, necessitating solutions for effective signal management. Consequently, restoring the signal via additional optical-electrical (OE/EO) conversions becomes essential, involving either the insertion of OE/EO converters along BDD edges [7] or interconnecting partitioned subcircuits with these converters [8], albeit at the cost of added delays. (Meanwhile, BDD variable ordering can be optimized for enhancing the signal efficiency [9].) To further improve the synthesized circuit, Zhao *et al.* explored the use of combiners and directional couplers (DCs) as WDM, while Matsuo *et al.* leveraged WDMs to reduce hardware costs [10], [11].

In order to avoid the timing delays caused by additional OE/EO converters, the signal magnitude should be maximized, and thus, the efficiency improvement methodology was proposed. The efficiency improvement methodology is defined by the engineering approaches in a PIC to modify its optical components without changing functionality, including switches, combiners, and detectors. In Fig. 2(a), *Efficiency factor* is used to measure loss on a reversed BDD described in Section II-B. In Fig. 2(b), a directional coupler (DC) combiner can replace a Y-branch combiner to improve loss described in Section II-A. Based on this information, Zhao *et al.* proposed two algorithms in the efficiency improvement methodology [6], which were performed on node i in Fig. 2. In Fig. 2(c), the Combiner Elimination algorithm duplicates a sub-BDD multiple times to eliminate that sub-BDD's sourcing combiner. In Fig. 2(d), the Coupler Assignment algorithm takes advantage of DC usage on two selected combiners whose parameters are obtained from quadratically constrained programming. There is still a room for further efficiency improvement. Therefore, in this paper, we design two novel algorithms in the efficiency improvement methodology based on the theory of harmonic mean and integer partition. In Fig. 2(e), the Efficiency Adjustment algorithm can achieve optimal efficiency by harmonic mean on the parameters of a DC combiner. In Fig. 2(f), the Switch Duplication algorithm

This work was supported in part by the National Science and Technology Council, Taiwan.

*Output Calculation in (a), (b)

(a): $a=b=c=d=e=1|0.8, f=0.8|0.64, g=\min(0.8,0.8,0.8)/3 \cdot 0.8=0.213, h=\min(0.64)/1 \cdot 0.64, i=\min(0.8,0.64,1,0.213)/4 \cdot 0.053, \text{ONE}=\min(0.053,0.8)/2 \cdot 0.027$
(b): Taking DC parameters (50%,50%,50%,250%) as an example, $i=\min(0.8 \cdot 50\%, 0.64 \cdot 50\%, 1 \cdot 50\%, 0.213 \cdot 250\%)/4 \cdot 0.080$. See Section II-A for details.

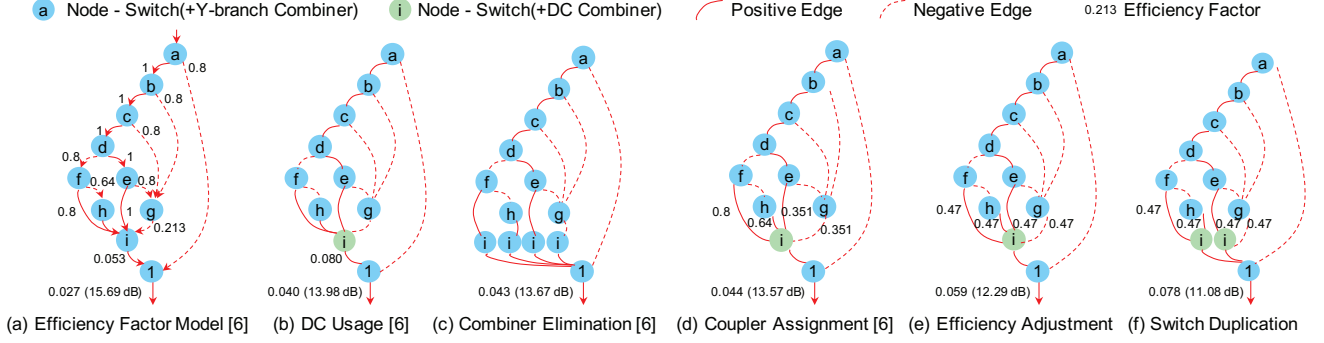


Fig. 2: Optical circuit synthesis. (a) Reverse BDD for the Boolean function $a' + abcd'fi + abcd'f'hi + abcdei + (ab' + abc' + abcde')g'i$. Proper switch output assignment results in an efficiency factor of 0.027. (b) A directional coupler combiner can replace the Y-branch combiner at node i to enhance BDD's efficiency (0.040). (c,d) [6] performs either Combiner Elimination or Coupler Assignment to improve efficiency. (e,f) Our algorithm achieves twofold improvement in efficiency.

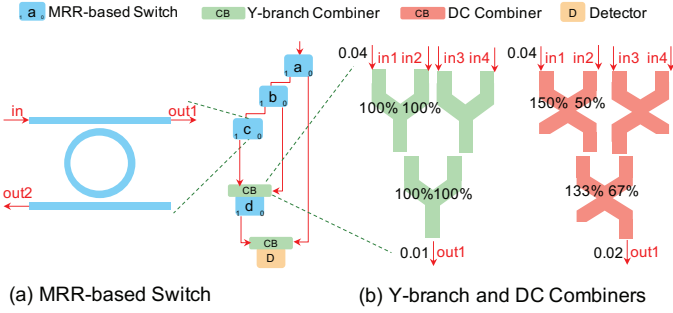


Fig. 3: Optical components of BDD synthesis. (a) An MRR-based switch with one input and two output directions. (b) A combiner is composed of multiple 2×1 combiners, either Y-branch combiners or directional coupler (DC) combiners.

traverses all switch duplication possibilities via integer partition and its factor permutations.

The remainder of this paper is organized as follows. Section II introduces the optical components, efficiency/loss model, and the addressed PIC logic synthesis problem formulation. Section III introduces the mathematical theories to design two algorithms and discusses their arrangement sequence. Section IV outlines our integrated framework. Section V demonstrates experimental results. Finally, Section VI concludes this work.

II. PRELIMINARIES AND PROBLEM FORMULATION

This section introduces the optical components, details the efficiency factor computation, and states the problem formulation.

A. Optical Components

Fig. 3 illustrates the three key optical components used for implementing a BDD.

1) *Detector*: The detector, an essential element in every PIC, transforms the signal back into an electronic signal. It can detect light above a certain threshold. As signal falls below this threshold, an OE/E0 converter will be added to restore power, which induces additional timing delay, hardware cost, and design complexity [7], [8]. Therefore, we should avoid OE/E0 converters by improving the signal magnitude of optical circuits as much as possible.

2) *Switch*: Research in [6], [10] demonstrates that microring-resonator-based (MRR) optical switches are capable of synthesizing abstract BDD nodes. An optical terminator effectively resolves garbage output issues. Within the MRR-based switch architecture, resonance is utilized to direct light towards the second output, causing a certain transmission loss, as depicted in Fig. 3(a). By default, the first output (out1) is designated to the BDD node's positive edge, while the second output (out2) is for the BDD node's negative edge (else-edge). The signal magnitude ratio between out2 and out1 is 0.8, corresponding to a transmission loss of approximately 1dB. Proper flipping the use of these two outputs may improve the signal efficiency, e.g., node g in Fig. 2. Other previous studies have incorporated different components. For instance, works by [4], [5] employed Mach-Zehnder interferometer (MZI), while [8] applied a DC for the BDD node.

3) *Combiner*: To aggregate incoming edges, combiners are designed as either Y-branch or directional coupler (DC) types. In Fig. 3(b), both N -input Y-branch combiners and DC combiners are in an array structure, and their efficiency degradation due to the mode mismatch ratio of 50% is calculated by $50\% \lg(N) = N^{-1}$ [6]. Using a DC combiner allows adjustments to the efficiencies of incoming edges, optimizing overall performance. Consider a set of input efficiencies, $in_1, \dots, in_N$, to an N -input DC combiner. Based on its functionality, we could adjust the input efficiencies by a scalar set, $k_1, \dots, k_N$:

$$(in_1, \dots, in_N) \rightarrow (k_1 in_1, \dots, k_N in_N) \text{ w.r.t. } \sum_{j=1}^N k_j = N,$$

with an additional area of $N - 1$ switches as hardware cost. For example, the scalar set of DC combiner at node i in Fig. 2(b) is $k_1, k_2, k_3, k_4 = 0.5, 0.5, 0.5, 2.5$; $0.5 + 0.5 + 0.5 + 2.5 = 4$. Interestingly, all of the aforementioned combiners can function as splitters via the reciprocal and the scatter matrix symmetry.

Based on the knowledge of these optical components above, we now introduce the efficiency factor model to estimate the loss in our work.

B. Efficiency Factor and Loss Model

Power loss in the synthesized PIC is estimated by calculating the efficiency factor of its BDD conversion, illustrated in Fig. 2(a) [6].

For each BDD, we compute the efficiency factor of each edge and node in topological order, denoted by γ : For edges,

the respective efficiency factor of a positive and negative edges is set to 1.0 and 0.8 due to the actual switch structure. For nodes, we consider the worst-case efficiency of their incoming edges. And, the efficiency $\gamma(in_1, \dots, in_N)$ can be formulated as below:

$$\text{Y-branch Combiner: } \gamma = \frac{\min_{j=1}^N \{in_j\}}{N},$$

$$\text{DC Combiner: } \gamma = \frac{\min_{j=1}^N \{k_j in_j\}}{N} \text{ w.r.t. } \sum_{j=1}^N k_j = N.$$

For example, a Y-branch combiner is applied to node i in Fig. 2(a), while a DC combiner is used in Fig. 2(b).

Overall, the efficiency of a BDD is defined by the efficiency factor of its 1-terminal, γ_{Term} ; the loss of a BDD is defined from its efficiency factor, $-10 * \log_{10}(\gamma_{\text{Term}})$. Also, the critical path within this BDD, from its source node to terminal one node exhibiting the worst-case (lowest) efficiency factor, is identified. Finally, a synthesized PIC's efficiency, loss, and critical path is the worst-case of efficiency, loss, and critical path to all BDDs.

C. Problem Formulation

To reduce power loss and elevate efficiency factors through logic synthesis, we address the following efficiency optimization problem:

The PIC Efficiency Optimization Problem: Given a circuit of Boolean functions and optical components, including switches, combiners, and detectors, the primary objective is to generate a logic circuit that maximizes the worst-case (lowest) efficiency factor, and the secondary objective is to minimize the number of additional OE/EO converters for power restoration.

III. THEORETICAL FOUNDATIONS

In this section, we first introduce the harmonic mean and integer partition optimization to combiners, followed by an analysis of their optimization order and the design of our main algorithm.

A. Harmonic Mean Optimization

Sections II-A and II-B introduce an efficiency factor model for an N -input DC combiner, defined as $\gamma = \frac{\min_{j=1}^N \{k_j in_j\}}{N}$, subject to the condition $\sum_{j=1}^N k_j = N$. However, this model presents significant challenges. The objective is to determine the optimal scalar set, $k_1, \dots, k_N$, which maximizes the efficiency factor of an N -input DC combiner, γ , called the **Efficiency Adjustment Problem**. Therefore, the harmonic mean is introduced as a solution method.

Theorem 1. Let us first define the the harmonic mean of N -inputs, $\text{Harm}(in_1, \dots, in_N) := \frac{N}{\frac{1}{in_1} + \dots + \frac{1}{in_N}}$. For an N -input DC combiner, the maximum value of the efficiency factor is equal to $\text{Harm}(in_1, \dots, in_N) * \frac{1}{N}$.

Proof. For a simpler representation, the numerator of the efficiency factor term is designated by f . Thus, $\gamma = \frac{f}{N}$ and $f(in_1, \dots, in_N) = \min_{j=1}^N \{k_j in_j\}$ w.r.t. $\sum_{j=1}^N k_j = N$. Our objective is to prove that the harmonic mean represents the maximum value of f :

$$f \leq k_j in_j \Leftrightarrow f * \frac{1}{in_j} \leq k_j, \quad \forall j = 1, \dots, N, \text{ w.r.t. } \sum_{j=1}^N k_j = N,$$

$$f * \left(\frac{1}{in_1} + \dots + \frac{1}{in_N} \right) \leq \sum_{j=1}^N k_j = N,$$

$$f \leq \frac{N}{\frac{1}{in_1} + \dots + \frac{1}{in_N}} = \text{Harm}(in_1, \dots, in_N),$$

the last equality holds if and only if $k_j = \frac{\text{Harm}(in_1, \dots, in_N)}{in_j}, \forall j = 1, 2, \dots, N$. Therefore, the maximum value of the efficiency factor is exactly $\text{Harm}(in_1, \dots, in_N) * \frac{1}{N}$. $\square$

Consequently, given an N -input DC combiner, we can calculate its maximum efficiency factor and corresponding scalar set in linear time complexity, called '**Efficiency Adjustment by Harmonic Mean**.' Now, we discuss another optimization issue.

B. Integer Partition Optimization

Expanding on prior research [6], the 'Combiner Elimination' algorithm eliminates an N -input combiner by applying an N -piece partitioning method, as shown in Fig. 2(c).

This action can be viewed as $N-1$ times duplication of a BDD switch node. An alternative approach is to duplicate the switch once, twice, thrice, etc. As a result, another problem arises. The objective is to enumerate all possible ways to duplicate switch nodes first and then arrange the incoming edges to the duplicated switches in order to optimize the efficiency factor, called the **Switch Duplication Problem**.

For the first objective, we propose a refined method using integer partition theory, which studies all possibilities for partitioning an integer number. For example, when $N = 4$ for node i in Fig. 2, the possible partitions include '4', '3 + 1', '2 + 2', '2 + 1 + 1', and '1 + 1 + 1 + 1'. Notably, the partition of '1 + 1 + 1 + 1' is exactly the special case corresponding to 'combiner elimination.' Typically, partition case enumeration can be executed using Depth-First Search (DFS), although this results in an exponentially large number of instances:

$$\frac{1}{4n\sqrt{3}} \exp(\pi\sqrt{\frac{2n}{3}}) \text{ as } n \rightarrow \infty, [12].$$

For the second objective, we introduce a greedy method for the optimal arrangement of input factors across all duplication possibilities. It can be formulated as follows:

The Factor Arrangement Subproblem: Suppose an N -input combiner, its input factors $in_1 \leq \dots \leq in_N$, and an integer partition of N , $N = p_1 + \dots + p_k$, where p_i is the number of factors required for the i -th switch. We aim to maximize the worst-case efficiency factor by arranging N -many incoming factors to k -many duplicated switches, that is:

$$\text{Through } \{in_1, \dots, in_N\} \Leftrightarrow IN_{p_1} \sqcup IN_{p_2} \sqcup \dots \sqcup IN_{p_k},$$

$$\text{To maximize } \min_{t=1}^k \frac{\min\{IN_{p_t}\}}{p_t},$$

where IN_{p_i} is the i -th partition of the set of input factors.

Theorem 2. In the **Factor Arrangement Subproblem**, arranging the lowest input factor to the smallest partition is always optimal.

Proof. Let us relabel $p_1 \leq \dots \leq p_k$ for easier description. It is always true that the smallest partition must contain the lowest input factor in the optimal arrangement solution.

Suppose not, we may assign in_1 to some IN_{p_t} ($t > 1$) to afford a larger partitioning efficiency factor. However, this assignment will break the lower bound to the worst-case efficiency factor:

$$\text{Suppose not: } \frac{\min\{in_1 \cup IN_{p_t}\}}{p_t} = \frac{in_1}{p_t},$$

$$\text{However: } \frac{\min\{IN_{p_1}\}}{p_1} = \frac{in_1}{p_1} \geq \frac{in_1}{p_t}, \frac{\min\{IN_{p_t}\}}{p_t} \geq \frac{in_1}{p_t}. \quad \text{---}$$

Leveraging the greedy property above, we can iteratively arrange the lowest input factor to the smallest partition to derive an optimal solution to the **Factor Arrangement Subproblem**. $\square$

Combining insights from integer partitioning, DFS enumeration, and optimal assignment solutions, we theoretically propose

the ‘Switch Duplication by Integer Partition’ algorithm. The subsequent section shows the optimization order to implement the ‘Efficiency Adjustment’ and ‘Switch Duplication’ algorithms.

C. Optimization Order

The optimal sequence for performing operations warrants investigation. Potential approaches include: (A) Efficiency Adjustment only, (B) Switch Duplication only, (C) Efficiency Adjustment followed by Switch Duplication, and (D) Switch Duplication followed by Efficiency Adjustment. Again, we can answer this question through a theoretical examination.

Theorem 3. *To enhance the worst-case efficiency factor of an N -input combiner, the optimal operational sequence is ranked as follows: option (C) supersedes option (A), which in turn outperforms option (D), expressed as ‘(C) $\succeq$ (A) $\succeq$ (D).’*

Proof. First, (C) $\succeq$ (A) trivially holds since the possibility of not duplicating the switch (not partitioning integer) is also considered in the Switch Duplication algorithm. This means that option (A) is considered within option (C).

Secondly, we prove (A) $\succeq$ (D) now. Suppose an N -input combiner, its input factors $in_1 \leq \dots \leq in_N$, and an integer partition of N , $N = p_1 + \dots + p_N$ w.r.t. $p_1 \leq \dots \leq p_N$, are given. Let us compare the numerators of the worst-case efficiency factors of (A) and (D), denoted by f and g respectively:

(D) - Duplicate Combiner first:

$$\begin{aligned} \{in_1, \dots, in_N\} &\xrightarrow{\text{Greedy Arrangement}} IN_{p_1} \sqcup IN_{p_2} \sqcup \dots \sqcup IN_{p_k}, \\ &\Leftrightarrow \{in_1, in_2, \dots, in_{p_1}\} \sqcup \{in_{p_1+1}, in_{p_1+2}, \dots, in_{p_1+p_2}\} \sqcup \dots \\ &\Leftrightarrow IN_{p_t} = \{in_{p_1+\dots+p_{t-1}+1}, \dots, in_{p_1+\dots+p_{t-1}+p_t}\}, \forall t = 1, \dots, k. \end{aligned}$$

(D) - Followed by Adjust Efficiency: $(\forall t = 1, \dots, k)$

Efficiency factor of IN_{p_t} : $\text{Harm}(IN_{p_t})$,

Worst-case efficiency factor: $g \leq \text{Harm}(IN_{p_t})$,

$$\begin{aligned} &\Leftrightarrow g \leq \frac{1}{\frac{1}{in_{p_1+\dots+p_{t-1}+1}} + \dots + \frac{1}{in_{p_1+\dots+p_{t-1}+p_t}}}, \\ &\Leftrightarrow \frac{p_t}{g} \geq \frac{1}{in_{p_1+\dots+p_{t-1}+1}} + \dots + \frac{1}{in_{p_1+\dots+p_{t-1}+p_t}}. \quad (*) \end{aligned}$$

(A) - Efficiency Adjustment only:

$$\begin{aligned} \text{Worst-case efficiency factor: } f &= \frac{N}{\frac{1}{in_1} + \dots + \frac{1}{in_N}}, \\ \Rightarrow f &= \frac{N}{\frac{1}{in_1} + \dots + \frac{1}{in_N}} \stackrel{(*)}{\geq} \frac{N}{\frac{p_1}{g} + \dots + \frac{p_k}{g}} = \frac{N}{\frac{p_1+\dots+p_k}{g}} = \frac{N}{\frac{N}{g}} = g. \end{aligned}$$

Hence, we derive that the best sequence of operations is (C) $\succeq$ (A) $\succeq$ (D). $\square$

Following this, the Switch Duplication algorithm could be simplified using the insights gained from above.

D. Simplified Switch Duplication

Adopting the strategy of ‘Efficiency Adjustment followed by Switch Duplication,’ it becomes possible to simplify the Switch Duplication algorithm using the information derived from the harmonic mean. For an adjusted N -input DC combiner, the **Factor Arrangement Subproblem** is no longer a difficulty since all input factors are $\text{Harm}(in_1, \dots, in_N)$. Furthermore, it can be proved that only evenly dispersed partitions can be the optimal partition. For $N = 6$, feasible partitions include ‘6’, ‘3 + 3’, ‘2 + 2 + 2’, ‘2 + 2 + 1 + 1’, ‘2 + 1 + 1 + 1 + 1’,

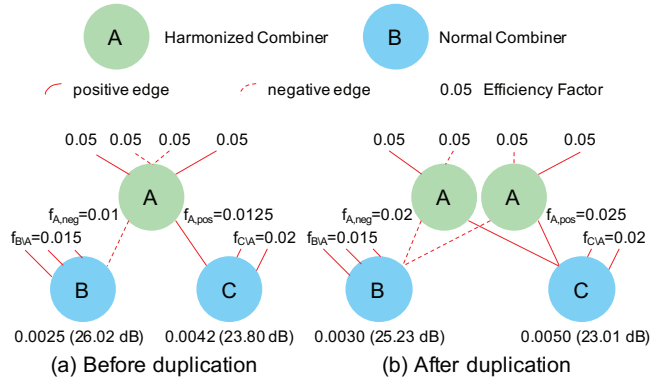


Fig. 4: In (a) is an adjusted 4-input DC combiner A, and its children switch nodes B and C. In (b), the Switch Duplication with respect to the 2 + 2 partition is considered. Moreover, the hardware costs before and after duplication are the same.

and ‘1 + 1 + 1 + 1 + 1 + 1’, excluding configurations such as ‘5 + 1’ or ‘3 + 2 + 1’. This breakthrough reduces the partition instances from exponentially to linear and allows for replacing DFS enumeration with simple linear traversal.

Fig. 4 illustrates the operation of a Simplified Switch Duplication corresponding to a ‘4 = 2 + 2’ partition. In Fig. 4(a), we observe that the input factors for the adjusted DC combiner A are uniform at 0.05, which acts as a lower bound on the efficiency of combiners B and C. In Fig. 4(b), through applying a ‘2 + 2’ partition on the adjusted DC combiner, the efficiency at combiners B and C is seen to increase. Moreover, the hardware costs before and after duplication are 4 + 1 + 1 and 2 + 2 + 1 + 1 switches, which are maintained. This case study demonstrates the significant impact of partitioning on not just the adjusted DC combiner but also on children combiners.

To formulate this tradeoff, the efficiency factors of combiner A and its two derivatives, combiner B and C, are considered. The efficiency factor before partition is noted as f_A , f_B , and f_C , respectively. After duplication with respect to the partition of $N_A = \frac{N_A}{k} + \dots + \frac{N_A}{k}$, we have:

$$\begin{aligned} f_A &= \frac{\text{Harm}(in_1, \dots, in_{N_A})}{N_A} \Rightarrow f'_A = \frac{\text{Harm}(in_1, \dots, in_{N_A})}{\frac{N_A}{k}}, \\ f_B &= \frac{\min(f_A, f_{B-A})}{N_B} \Rightarrow f'_B = \frac{\min(f'_A, f_{B-A})}{N_B - 1 + k}, \\ f_C &= \frac{\min(f_A, f_{C-A})}{N_C} \Rightarrow f'_C = \frac{\min(f'_A, f_{C-A})}{N_C - 1 + k}, \end{aligned}$$

$$\text{Objective function: } \log\left(\frac{f'_B}{f_B}\right) + \log\left(\frac{f'_C}{f_C}\right),$$

where N_A , N_B , and N_C are the number of incoming edges of A, B, and C, respectively. To leverage the benefits of Switch Duplication effectively, we introduce normalization as a core concept to establish an objective function: $\log\left(\frac{f'_B}{f_B}\right) + \log\left(\frac{f'_C}{f_C}\right)$. The aim is to select the partition that maximizes the objective function output in a Switch Duplication scenario. It is worth mentioning that in the base case, when $k = 1$, the function’s output is precisely 1. The following work calls this algorithm ‘Simplified Switch Duplication.’

IV. OUR SYNTHESIS FRAMEWORK

In this section, we present our PIC logic synthesis framework, named Harrow.

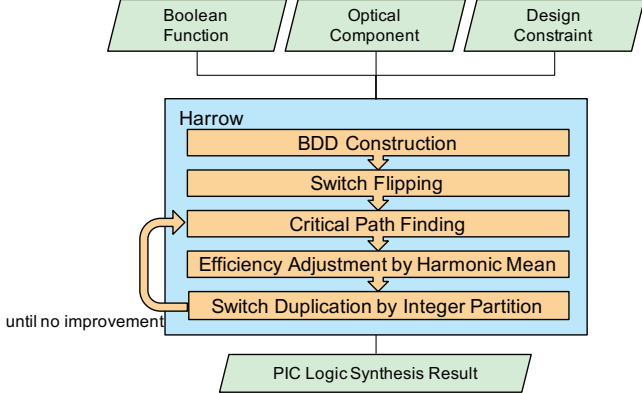


Fig. 5: The overall flow of Harrow.

Algorithm 1: Harrow

```

1 while efficiency has not converged do
2    $(f, n_S, n_T) \leftarrow$  the BDD, its source, and terminal-one
3   for each node  $n_i$  in  $f$ , from  $n_T$  to  $n_S$  do
4      $\text{swap}(n_i.\text{left\_child}, n_i.\text{right\_child})$ 
5      $n_i \leftarrow n_i.\text{argmin}(in_1, \dots, in_N)$ 
6 while  $\text{ans} == \text{True}$  do
7    $(f, n_S, n_T) \leftarrow$  the BDD, its source, and terminal-one
8    $\text{ans} = \text{Combiner-Optimization}(f, n_S, n_T)$ 

```

Algorithm 2: Combiner-Optimization

```

1  $(f, n_S, n_T) \leftarrow$  the chosen BDD, its source, and terminal-one
2 for each node  $n_i$  in  $f$ , from  $n_T$  to  $n_S$  do
3   if  $n_i.\text{inputs} < 2$  then
4     continue
5   if  $n_i.\text{Adjusted}$  and  $n_i.\text{Duplicated}$  then
6     continue
7   if not  $n_i.\text{Adjusted}$  then
8      $\text{Efficiency\_Adjustment}(n_i)$ 
9      $n_i.\text{Adjusted} = \text{True}$ 
10  else if not  $n_i.\text{Duplicated}$  then
11     $(A, B, C) \leftarrow (n_i, n_i.\text{left\_child}, n_i.\text{right\_child})$ 
12    for each evenly dispersed partition of  $n_i$  do
13       $\text{Compute } f'_A, f'_B, f'_C \text{ and } \text{Obj}(A, B, C)$ 
14       $\text{Switch\_Duplication}(n_i)$ 
15      // Through the partition with largest Obj value
16       $n_i.\text{Duplicated} = \text{True}$ 
17  return True
18   $n_i \leftarrow n_i.\text{argmin}(in_1, \dots, in_N)$ 
19 return False

```

A. Overview

Fig. 5 depicts the overall flow of Harrow. First, we construct an abstract BDD from the given Boolean function. Subsequently, we properly flip the switch output assignment of positive and negative edges on the generated BDD to enhance loss. The framework proceeds with iterative identification of the critical path, wherein Efficiency Adjustment and Switch Duplication algorithms are applied to the respective BDD nodes, repeating the process until convergence is achieved with no further optimization.

TABLE I: Performance Scalability Comparison of net efficiency improvement (Imprv) against limited/actual switch ratio cost (Limit/#SW) for [5], [6], Harrow with Efficiency Adjustment only (Harrow-EA), and proposed Harrow (Harrow-All).

Limit	[5]		[6]		Harrow-EA		Harrow-All	
	#SW	Imprv	#SW	Imprv	#SW	Imprv	#SW	Imprv
1	1.000	1.000	1.000	8.965	1.000	7.853	1.000	7.853
1.05	1.000	1.000	1.015	14.702	1.025	9.070	1.024	9.092
1.1	1.000	1.000	1.046	18.457	1.057	11.209	1.057	11.206
1.15	1.000	1.000	1.069	18.646	1.091	16.913	1.091	16.652
1.2	1.000	1.000	1.089	20.976	1.123	29.855	1.126	24.897
1.25	1.000	1.000	1.111	18.678	1.158	42.992	1.158	33.330
1.3	1.000	1.000	1.125	20.372	1.187	79.171	1.191	76.921
1.35	1.000	1.000	1.138	18.746	1.208	99.493	1.215	92.678
1.4	1.000	1.000	1.153	19.171	1.235	121.999	1.242	133.731
1.45	1.000	1.000	1.172	20.935	1.258	143.358	1.263	147.941
1.5	1.000	1.000	1.199	20.894	1.280	164.992	1.288	170.084
1.55	1.000	1.000	1.216	21.940	1.295	167.468	1.302	171.198
1.6	1.000	1.000	1.230	20.519	1.316	169.129	1.328	174.732
1.65	1.000	1.000	1.242	21.252	1.339	174.116	1.354	178.601
1.7	1.000	1.000	1.261	22.667	1.361	176.312	1.378	182.183
2	1.000	1.000	1.345	24.132	1.391	187.084	1.411	192.875

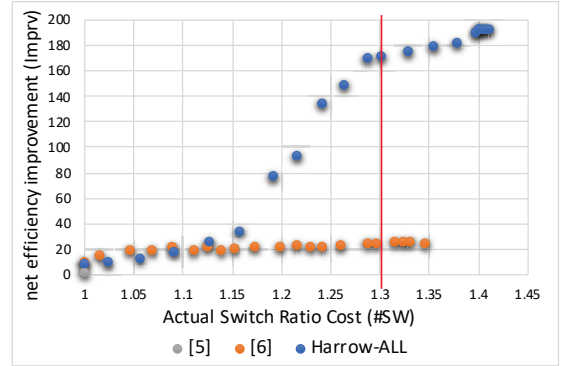


Fig. 6: Visual representation of Table I. Performance Scalability Comparison of net efficiency improvement (Imprv) against actual switch ratio cost (#SW) for [5], [6], and proposed Harrow (Harrow-All).

B. Switch Flipping

Let us consider a BDD generated from the given single-bit Boolean function. As mentioned in Section II-A2, we can improve the efficiency by flipping the positive and negative edge output of a BDD switch node, corresponding to a (1.0, 0.8) exchange of the efficiency factor. The Switch Flipping is to flip BDD nodes located along the critical path. Empirical evidence indicates that the BDD's efficiency typically stabilizes after approximately a dozen iterations of switch flipping.

C. Critical Path Finding

To identify the critical path, one observation is the critical path could be dynamically changed during the BDD efficiency optimization process. Therefore, it is necessary to identify the critical path for each time. Critical Path Finding involves traversing the BDD from the terminal node back to the source node, iteratively identifying and backtracking through parent nodes with the minimum incoming efficiency, denoted as $\text{argmin}(in_1, \dots, in_N)$. To analyze the time complexity of this algorithm, the maximum height equates to the variable count for any BDD.

D. Combiner Optimization

The Combiner Optimization includes Efficiency Adjustment by Harmonic Mean and Switch Duplication by Integer Partition, as depicted in Algorithm 2. In lines 1–18, we may leverage

TABLE II: Comparison of net efficiency improvement (Imprv), loss (Loss) (dB), and CPU time (Time) (s) for [5], [6], Harrow-EA, and Harrow-All at actual switch ratio cost (#SW) of 1.3.

Benchmark	#PI	#PO	[5]				[6]				Harrow-EA				Harrow-All			
			#SW	Imprv	Loss	Time	#SW	Imprv	Loss	Time	#SW	Imprv	Loss	Time	#SW	Imprv	Loss	Time
cps	24	109	1654	1.00	32.35	0.10	2219	23.10	23.82	0.29	2079	91.87	18.10	0.10	2082	91.87	18.10	0.11
alu4	14	8	653	1.00	45.56	0.06	814	11.11	37.08	0.12	976	49.70	32.50	0.06	961	27.22	33.87	0.06
table3	14	14	1634	1.00	42.29	0.06	2036	11.51	32.68	0.08	2083	93.26	24.55	0.07	2085	93.65	24.53	0.07
apex5	117	88	629	1.00	27.96	0.11	724	2.33	25.46	0.14	868	4.07	22.62	0.11	834	4.89	21.63	0.11
table5	17	15	1757	1.00	48.19	0.06	2276	65.15	33.89	0.09	2315	513.25	25.85	0.07	2347	545.03	25.71	0.13
vda	17	39	1017	1.00	28.76	0.07	1347	3.77	23.89	0.09	1437	12.85	18.79	0.07	1450	13.02	18.75	0.08
k2	45	45	2009	1.00	41.98	0.09	2652	66.59	29.85	0.11	2655	933.57	22.26	0.09	2679	958.94	22.15	0.12
seq	41	35	2192	1.00	44.09	0.09	2818	46.31	31.94	0.15	2671	382.58	24.69	0.10	2718	399.76	24.54	0.13
count	35	16	195	1.00	28.27	0.06	312	35.97	14.54	0.06	220	13.72	17.55	0.06	220	13.72	17.55	0.11
dalv	75	16	1482	1.00	44.58	0.10	1817	8.62	37.63	0.12	1771	50.87	30.30	0.10	1859	53.44	30.12	0.11
i2c	147	142	947	1.00	30.46	0.25	1181	38.08	20.60	0.29	1150	100.09	15.60	0.26	1154	98.85	15.64	0.30
i8	133	81	433	1.00	32.49	0.12	564	5.16	26.74	0.14	581	50.98	19.08	0.13	599	52.08	19.01	0.13
simple_spi	148	144	763	1.00	27.13	0.26	961	8.42	21.56	0.29	939	22.52	17.27	0.26	942	22.10	17.31	0.29
frg2	143	139	708	1.00	37.27	0.16	933	7.10	30.70	0.19	948	25.22	24.98	0.16	947	22.22	25.30	0.18
Comparison	-	-	1.000	1.000	-	-	1.297	23.873	-	-	1.295	167.468	-	-	1.302	171.198	-	-

the Critical Path Finding to traverse the BDD in search of nodes to apply the Efficiency Adjustment or Switch Duplication algorithms. Lines 10–16 integrate the methodology described in Section III-D to implement the **Simplified Switch Duplication algorithm** effectively.

Now, we can describe our Harrow as listed in Algorithm 1. Lines 1–5 employ the Switch Flipping algorithm to enhance efficiency, while lines 6–8 iteratively apply the Combiner Optimization until the process indicates no further potential for improvement.

V. EXPERIMENTAL RESULTS

The proposed algorithm and the methods from [5], [6] were implemented in C++ programming language and integrated with the ABC engine [13]. All experiments were performed on an Apple M1 3.2GHz MacBook Pro with 16GB RAM. Consistent with previous works [6], [7], [10], we used the benchmark [14] in our experiment. In the experiment, the execution commands involved: ‘read benchmark,’ ‘collapse,’ ‘Extra_ReorderCudd,’ and a custom command of experimental algorithms. Certain benchmarks could not be tested: ‘stpmotor (stepper motordrive),’ ‘spla,’ ‘alu4,’ and ‘misex3c’ cases.

Table I provides a comparative analysis of the scalability of our proposed algorithms against [5], [6], highlighting significant improvements in net efficiency across various switch ratio limits. Since the detection threshold value was set to 15dB, 20dB, 25dB, and 40dB in previous works [7], [8], we only measured these efficiency improvement algorithms on BDDs whose terminal loss exceeded 15dB. The first line is the upper bound of the switch ratio cost (Limit) to every measured BDD. For these four methods, columns 2, 4, 6, 8 are the exact switch ratio cost, and columns 3, 5, 7, 9 are the average of measured BDDs’ net efficiency improvement on these 14 benchmarks. For example, if the switch ratio cost is limited to no more than 1.5 on every BDD of benchmarks, the actual switch ratio cost of these four methods overall is 1.000, 1.199, 1.280, 1.288. Fig. 6 visualizes the scalability of net efficiency improvements with increasing switch ratio costs, demonstrating the superior performance of our proposed approach. The switch ratio cost of 1.3 represents an optimal balance between performance improvement and switch count cost, serving as a sweet point for algorithm scalability.

Table II indicates the effectiveness of our designed algorithms, benchmarking our results against [5], [6] when the switch ratio cost is restricted to 1.3. The initial three columns list the benchmark information, including the number of primary inputs (#PI) and outputs (#PO). For these four methods, columns

4, 8, 12, 16 are the exact switch ratio cost (#SW) on each benchmark; columns 5, 9, 13, 17 are the average of net efficiency improvement (Imprv) on each benchmark; columns 6, 10, 14, 18 outline the loss, which is calculated by $-10 * \log_{10}(\gamma_{Term})$, see Section II-B; columns 7, 11, 15, 18 list the CPU times in seconds. All experimental algorithms were performed independently on the measured BDDs of benchmarks in the entire experiment. For [6], iterations of combiner elimination and coupler assignment were set to 5 and 37, respectively. For our designed methods, Harrow-EA and Harrow-All, the iterations of switch flipping were set to 50, and the maximum partition count (k) was 1 and 6, respectively.

First of all, we compared [6] and Harrow-All. Our Harrow-All method achieved a double-digit net efficiency improvement over [6], with a seven times improvement and better loss on 13 of 14 benchmarks. However, when the switch ratio cost was limited to 1.05 or 1.1, Harrow-All underperformed due to insufficient switch quota for the ‘Efficiency Adjustment’ algorithm on small BDDs. Secondly, we compared Harrow-EA and Harrow-All. While Harrow-EA and Harrow-All performed better on 5 and 9 benchmarks, respectively, with average improvements tied for both. Our explanation is that despite the theoretical guarantee of no extra switch cost in a ‘Switch Duplication’ operation, the duplicated switches contribute additional edges, ultimately increasing the switch count and diminishing Harrow-All’s performance. In summary, all three methods demonstrated the potential to enhance net efficiency, and our Harrow-All method indicated a substantial advancement in optimizing PIC logic synthesis.

VI. CONCLUSIONS

To mitigate the signal attenuation issue in PICs, this paper presents three novel techniques for signal efficiency improvement, including harmonic mean, integer partition, and optimal operational sequence. The harmonic mean provides an optimal method for adjusting combiner efficiency, the integer partition offers a systematic approach to exploring all switch-duplication possibilities, and the optimal operational sequence ensures the efficient arrangement of these techniques. Together, these techniques form the basis for the Efficiency Adjustment and Simplified Switch Duplication algorithms, advancing the field of PIC logic synthesis. Experimental results demonstrate that Harrow achieves a double-digit improvement in efficiency compared to state-of-the-art methods, while maintaining comparable hardware overhead.

REFERENCES

- [1] C. Sun, M. T. Wade, Y. Lee, J. S. Orcutt, L. Alloatti, M. S. Georgas, A. S. Waterman, J. M. Shainline, R. R. Avizienis, S. Lin *et al.*, "Single-chip microprocessor that communicates directly using light," *Nature*, vol. 528, no. 7583, pp. 534–538, 2015.
- [2] A. Mirza, S. M. Avari, E. Taheri, S. Pasricha, and M. Nikdast, "Opportunities for cross-layer design in high-performance computing systems with integrated silicon photonic networks," in *Design, Automation and Test in Europe Conference (DATE)*, 2020, pp. 1622–1627.
- [3] P. Wesling, *Heterogeneous Integration Roadmap 2023*, 2023, Chapter 9: Integrated Photonics. [Online]. Available: https://eps.ieee.org/images/files/HIR_2023/ch09_photonics.pdf
- [4] C. Condrat, P. Kalla, and S. Blair, "Logic synthesis for integrated optics," in *Great Lakes Symposium on VLSI (GLSVLSI)*, 2011, pp. 13–18.
- [5] R. Wille, O. Keszocze, C. Hopfmüller, and R. Drechsler, "Reverse BDD-based synthesis for splitter-free optical circuits," in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2015, pp. 172–177.
- [6] Z. Zhao, Z. Wang, Z. Ying, S. Dhar, R. T. Chen, and D. Z. Pan, "Logic synthesis for energy-efficient photonic integrated circuits," in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2018, pp. 355–360.
- [7] Z. Zhao, J. Gu, Z. Ying, C. Feng, R. T. Chen, and D. Z. Pan, "Design technology for scalable and robust photonic integrated circuits," in *International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–7.
- [8] R. Matsuo, J. Shiomi, T. Ishihara, H. Onodera, A. Shinya, and M. Notomi, "A synthesis method for power-efficient integrated optical logic circuits towards light speed processing," in *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2020, pp. 488–493.
- [9] R. Matsuo and S.-i. Minato, "BDD variable ordering for minimizing power consumption of optical logic circuits," in *IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2021, pp. 96–101.
- [10] Z. Zhao, D. Liu, Z. Ying, B. Xu, C. Feng, R. T. Chen, and D. Z. Pan, "Exploiting wavelength division multiplexing for optical logic synthesis," in *Design, Automation and Test in Europe Conference (DATE)*, 2019, pp. 1567–1570.
- [11] R. Matsuo, J. Shiomi, T. Ishihara, H. Onodera, A. Shinya, and M. Notomi, "BDD-based synthesis of optical logic circuits exploiting wavelength division multiplexing," in *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2019, pp. 203–209.
- [12] G. E. Andrews, *The theory of partitions*. Cambridge university press, 1998, no. 2.
- [13] R. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *International Conference on Computer Aided Verification (CAV)*, 2010, pp. 24–40.
- [14] A. Mishchenko, "IWLS 2005 benchmarks," Accessed: 2024-06-16. [Online]. Available: <https://people.eecs.berkeley.edu/~alanmi/benchmarks/iwls2005/>